21/03/2022

COLLECTIONS FRAME WORK

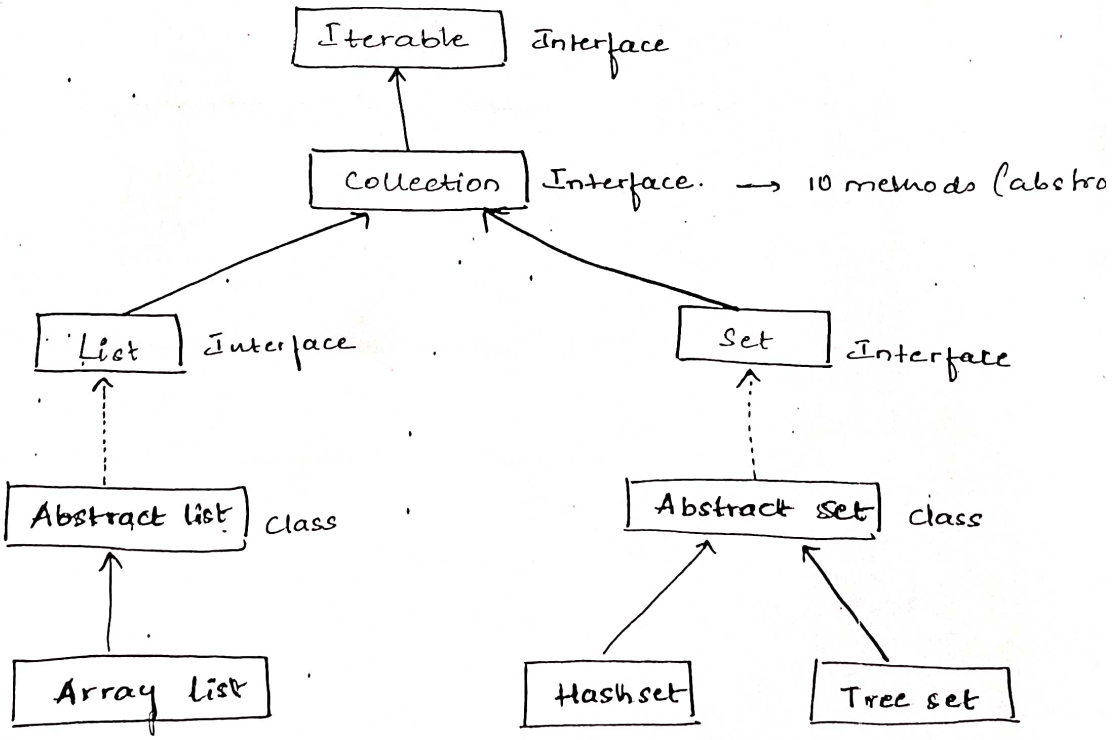
Collection frame work is a set of class & interface which provides mechanism to store groups of objects together.

It also provide built-in mechanism to perform all the create/ read /update/ delete · updation

NOTE: Two important Heirarchy of collection frame work are:

1. Collection Heirarchy.
2. Map Heirarchy.

Collection Heirarchy:



COLLECTIONS :

Collection is an interface which is present inside the package called as java.util package

Collection provide mechanism to store group of object together.

Inside collection all the elements are stored as Objects only [ie only Non-primitive are allowed to store

Collection have a few methods which helps the programmer to perform following task :

a. To add an element into the collection.

b. Search an element in the collection.

c. Remove an element from the collection.

d. To access an element from the collection.

METHODS OF COLLECTION:

SLNO	PURPOSE	SIGNATURE OF METHOD	RETURN TYPE
1	To add an element	add(Object o)	boolean
		addAll(Collection c)	boolean
2	To search an element	contains(Object o)	boolean
		containsAll(Collection c)	boolean
3.	To remove an element	remove(Object o)	
		removeAll(Collection c)	
		retainAll(Collection c)	
		clear()	
4.	To access	Iterator()	
		Access using foreach loop	
5.	Miscellaneous	size()	int
		isEmpty()	boolean
		toArray()	
		equals()	
		hashCode()	

LIST INTERFACE:

List Interface is present inside java.util package.

Since it is a sub-interface of collection, it has all the methods that are present in collection.

▶ CHARACTERISTICS OF LIST :-

1) List is a collection of object.

2) List follows the Order of Insertion [order in which we inserted the objects will be maintained]

3) List can have duplicate elements.

4) We can access the object in the list with the help of index. Therefor, we can add, remove, access anywhere with the help of index.

▶ METHODS OF LIST :-

SLNO	PURPOSE	SIGNATURE OF METHOD	RETURN TYPE
1.	To add an element	add (Object o) addAll (Collection c) add (int index, Object o) add (int index, Collection c)	
2.	To search an element	contains (Object o) containsAll (Collection c). indexOf (Object o)	
3.	To remove an element	remove (Object o) removeAll (Collection c) retainAll (Collection c) clear() remove (int index)	
4.	To access	iterator() listIterator() get() Access using foreach loop	
5.	Miscellaneous	size() isEmpty() toArray() equals() hcode()	

ARRAYLIST:

* Arraylist is a concrete Implementing class of List interface
* Arraylist is used to store the object in the form of List

CHARACTERISTIC OF ARRAY LIST:

* In Arraylist, the order of insertion is maintained.
* Arraylist can accept duplicate values.
* We can access the object in Arraylist with the help of index. Therefore we can add, remove or access anywhere with the help of index.

NOTE: 1) All the abstract methods of list & collection interfaces will be implemented inside Arraylist class.

2) We cannot create an Object for List, but we can instantiate Arraylist.

CONSTRUCTOR OF ARRAYLIST:

1) Arraylist() : It creates an empty array list object

2) Arraylist (Collection c) : It creates an Arraylist object & copies all the elements present in given collection into Arraylist.

3)

NOTE: Inside Arraylist, the tostring() method is overriden.

Ex for isEmpty() & size():

(i) **isEmpty():** It is used to check whether the collection empty or not. If the collection is emphy, it returns tr else it returns false.

ii) **size():** It gives us the no. of elements present ins the collection. The return type of size method is int.

Ex:
```
import java.util.ArrayList;
public class AL2{
    public static void main(String[] args)
    {
      Arraylist ls= new Arraylist();
      System.out.println(ls.isEmpty()); //true
      System.out.println(ls.size()); //0.
    }
}
```

The drawbacks of Arrays such as fixed size, Homogenou mixture can be overcome with the help of collections a the collection size can be modified dynamically as well as w can have any type of data stored inside one single collect as shown below:-

Ex:
```
class Pen
{
}
class Book { }
class Student { }
```

```
class AL3
{
 public static void main (String[] args)
 {
   ArrayList ls = new ArrayList();
    ls.add(10);
    ls.add(10.5);
    ls.add('A');
    ls.add("sheela");
    ls.add(true);
    ls.add(new Book());
    ls.add(new student());
    ls.add(new Scanner(System.in));
    ~~ls.add~~
    System.out.println(ls);
 }
}
```

olp:

[10, 10.5, A, sheela, true, Book@...., student@...., ...]

▶ add(Object o):

* It is used to add an element into the collectio
* It has the object type formal argument because to enable the user to pass whatever he wants.
* The return type of add(Object o) method is boolean
* Any element passed to the add method, will be stored as an Object type element.

Ex:
```
import java.util.ArrayList;
class A15
{
 public static void main (String[] args)
 {
   ArrayList ls = new ArrayList();
   ls.add(10);
   ls.add(20);
   ls.add(30);
   ls.add(20);
   S.o.ptln(ls);
 }
}
```

o/p:

[10, 20, 30, 20]

WASP to design a method which returns the collection of prime numbers from m to n. [The range between m & n must be atleast 50]

```
import java.util.ArrayList;
class prime
{
 public static ArrayList prime(int m, int n)
 {
  ArrayList ls = new Array list();
  while (m<n)
  {
    int count = 0;
    for(int i=2; i<=m/2; i++)
    {
      if (m%i==0)
      {
        count++;
        break;
      }
```

```
if (count==0)
{
  ls.add(m);
}
m++;
}
return ls;
}
public static void main(String[] args)
{
  Arraylist ls= prime(1,50);
  System.out.println(ls);
}
}
```

▶ addAll(Collection c) :-

- addAll method accepts collection as input.
- whenever we pass a collection for addAll method, the pass collection will not be added as a single object. Instead, each element of it will be added to the referred collection.

Eg:

```
class A17
{
  public static void main(String[] args)
  {
    Arraylist bag1= new Arraylist();
    bag1.add(new Book1("Manual"));
    bag1.add(new Book1("Java"));
    System.out.println(bag1);
  }
```

```
Arraylist bag2 = new Arraylist();
bag2.add(new Book1("SQL"));
bag2.add(new Book1("selenium"));
System.out.println("bag2 :" + bag2);

bag1.add(bag2);
System.out.println("bag1 :" + bag1);
System.out.println(bag1.size());
bag1.addAll(bag2);
System.out.println("bag1 :" + bag1);
System.out.println(bag1.size());
}
}
class Book1
{
String title;
Book1(){}
Book1(String title)
{
this.title = title;
}
}
```

Op:

bag1 : [Manual, Java]

bag2: [SQL, selenium]

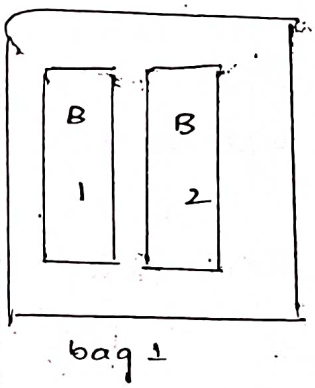
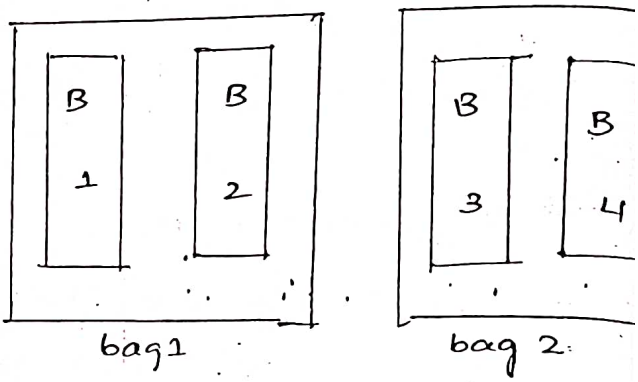
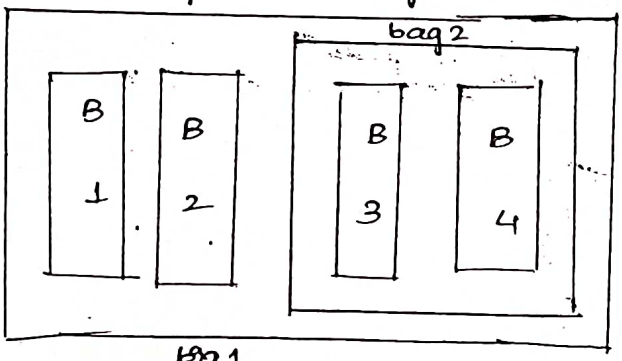
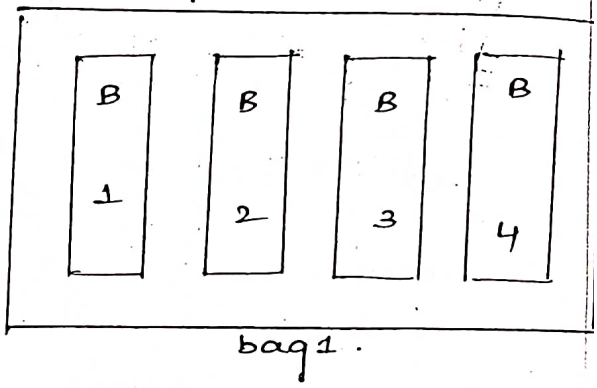
bag1: [Manual, Java, [SQL, selenium]]

3

bag1: [Manual, Java, SQL, selenium]

4.

DIFFERENCE Blw add(Object o) & addAll(Collection c)

add(Object o)	addAll(Collection c)
	
bag1.add(bag2) 	bag1.addAll(bag2) 

add(int index, Object o):

It is used to add the passed element at a particul index / specified index.

Ex:

```
import java.util.ArrayList;
class A10
{
  public static void main(String[] args)
  {
    ArrayList ls = new ArrayList();
    ls.add(10);
    ls.add(20);
    ls.add(30);
    ls.add(2,25);
```

```
System.out.println (ls.size());
System.out.println (ls);
```

Op: 4
[10, 20, 25, 30]

addAll (int index, Collection c):

It adds all the elements of passed collection from specified index inside referred collection.

Ex:
```
class A18
{
  public static void main (String[] args)
  {
    ArrayList bag1 = new ArrayList();
    bag1.add (new Book1 ("Manual"));
    bag1.add (new Book1 ("Java"));

    ArrayList bag2 = new ArrayList();
    bag2.add (new Book1 ("SQL"));
    bag2.add (new Book2 ("Selenium");

    bag1.addAll (1, bag2);
    System.out.println (bag1.size());
    System.out.println (bag1);
  }
}
```

Op:
4
[Manual, SQL, Selenium, Java]

ACCESSING THE ELEMENTS FROM COLLECTION :

The elements inside Array list can be accessed using following ways :-

1) get()
2) iterator()
3) listIterator()
4) for each loop.

get(index) :

i) get() is used to access the elements present in the array list.

ii) It accepts index as an argument

iii) Its return type is object type.

Eg

```
import java.util.ArrayList;
class Book1
{
int bid;
String title;
Book1() { }
public Book1(int bid, String title)
{
this.bid = bid;
this.title = title;
}
```

```
public String toString()
{
  return " Book1 [ bid= " + bid + ", title = " + title + " ] ";
}
}

class G4
{
  public static void main (String [] args)
  {
    ArrayList ls = new ArrayList();
    ls.add (new Book1 (101, "Manual"));
    ls.add (new Book1 (102, "SQL"));
    ls.add (new Book1 (103, "Java"));
    ls.add (new Book1 (104, "Selenium"));
    System.out.println (ls);
  // to remove the book whose name is Java
    for (int i=0; i< ls.size(); i++)
    {
      Book1 b1 = (Book1) ls.get(i);
      if (b1.title.equalsIgnoreCase ("Java"))
      {
        ls.remove (b1);
      }
    }
    System.out.println (ls);
  }
}
```

iterator():

To access the elements using iterator():

i) iterator is a method which belongs to Iterable interface

ii) iterator() creates a Iterable type object & return its reference.

iii) iterator() returns the reference in iterator type.

Iteration: repeating same activity.

Construction: Process of constructoing → constructor: person who construct.

Iterator: A person who iterate.

Iterator interface:

Iterator is an interface which is present inside java.util package

Iterator is an interface, which provides a mechanism to access the elements of collection

iterator() creates an instance of iterator interface and return its reference.

Inside iterator interface we have following methods which helps the user to access the elements

Public Boolean hasNext():

It checks whether is an element to be accessed from the collection. If present it returns true, else it returns false.

public Object next():

It is used to access the elements & it moves the cursor to next pointer. The return type of it will be Object type

Eg. class IT2
{
public static void main (String[] args)
{
ArrayList ls = new ArrayList();
ls.add(10);
ls.add(20);
ls.add(30);
System.out.println ("Before accessing" + ls);
Iterator i = ls.iterator();
while (i.hasNext())
{
System.out.println (i.next());
}
}
}

WAJP to add 5 markers into a collection & print how many markers are having same / how many markers are having the price 20.

1) We can access the elements of collections only once for a Iterator method call.

2) We can't access the elements of collection in reverse directi can't access the previous elements as there is no method to

[NoSuchElementException: Exception we get if we try to access the e which is not present.

IndexOutOfBoundsException: If we pass/try to access index which is gre than size of collection or less than 0]

▶ listIterator():

listIterator() is a non-static method which is present inside List interface

It creates an instance of ListIterator type & it returns the reference of java.util.ListIterator interface ie return type of listIterator() is ListIterator.

▶ ListIterator:

1) ListInterface is a sub interface (child) of Iterator interface.

2) It is present inside java.util package

▶ METHODS OF ~~LISTITERATOR~~ ListIterator:

▶ **boolean hasNext():** To check whether next element to be accessed is present or not

- **Object next()**: It gives the element & moves the cursor to next pointer.
- **boolean hasPrevious()**: To check whether previous element to be accessed is present or not.
- **Object previous()**: It gives the previous element pointed by the cursor & moves the cursor backwa
- **void remove()**: It removes the current Object pointe from the collection which is under iteration.
- **void add(Object o)**: It is used to add a new Object into the collection
- **void set(Object o)**: It is used to replace the elem of Collection.

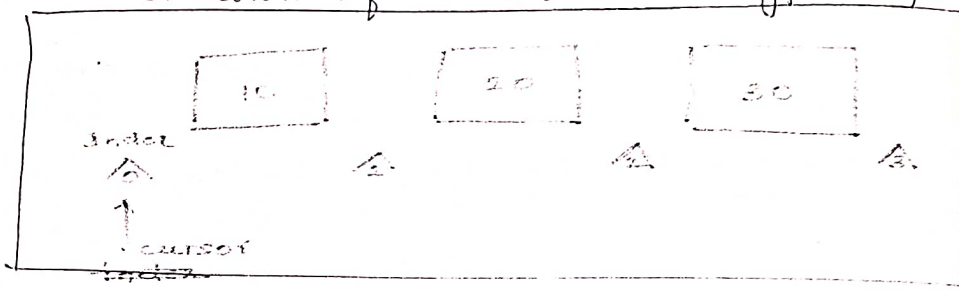
NOTE: listIterator() is an overloaded method in java.

=> **listIterator()**: It creates listIterator type object & the cursor will be pointing to the first index of the collection ie index '0'.

=> Eg:
```
import java.util.ArrayList;
class I3
{
  public static void main(String[] args)
  {
    ArrayList ls = new ArrayList();
```

```
ls.add(10);
ls.add(20);
ls.add(30);
System.out.println(ls);
ListIterator l = ls.listIterator();
while(ls.hasNext())
{
   l.next();
}
while(ls.hasPrevious())
{
   System.out.println(ls.previous());
}
```

Creation of listIterator type objec




```
}
}
```

o/p:

[10, 20, 30]

2> listIterator(int index):

It Creates listIterator type object & cursor will be pointing ~~towards~~ to the index specified.

Eg:

```
import java.util.ArrayList;

class I4
{
  public static void main(String[] args)
  {
    ArrayList ls = new ArrayList();
    ls.add(10);
    ls.add(20);
    ls.add(30);
    System.out.println(ls);
    ListIterator l = ls.listIterator(ls.size());
    while (l.hasPrevious())
    {
      System.out.println(l.previous());
    }
  }
}
```

o/p:

[10, 20, 30]



* Access using for each loop :

SYNTAX :

```
for (type of variable : ref.variable of collection/Array)
{
    // statements ;
}
```

Eg:

```
import java.util.ArrayList;
public class F1
{
  public static void main (String[] args) {
  ArrayList ls = new ArrayList();
  ls.add(10);
  ls.add(20);
  ls.add(30);
  System.out.println(ls);
  //access using advanced for each loop.
  for (Object o : ls)
  {
    System.out.println(o);
  }
}
```

(or)

```
{
  Integer i = (Integer)o;
  S.o.p(i);
}
```

Eg2: To access elements of array :-

```
class F2
{
public static void main (String[] args)
 {
  int[] a = {10, 20, 30};

 for(int i : a)
  {
   System.out.println(i);
  }
 }
}
```

NOTE : In java we have two types of Collection

- Non-generic Collection
- Generic collection.

NON-GENERIC COLLECTION:

It is a heterogenous (Different types) collection of elements. Every element is converted & stored as java.lang.Object class type.

GENERIC COLLECTION:

It is a homogenous collection of elements. (collection of same types of objects).

SYNTAX:

1. Syntax to create ref variable for generic collection:

```
Collectiontype<Non-primitive data type> identifier;
```

Ex:

ArrayList ls; → Non-generic ref variable creation.

Arraylist<Integer> ls; →

2. Syntax to create generic collection object:

```
new Collectiontype<Nonprimitive datatype>();
```

Eg: ~~A~~ new ArrayList<Integer>();

Eg: ArrayList<Integer> ls = new ArrayList<Integer>();

This is a collection of ArrayList inside which I can store only Integer values.

POINT TO BE REMEMBERED: From JDK 1.7 onwards, Syntax to create generic collection is:

```
Collectiontype<Non-primitive type> ref.var = new Collect.type<>()
```

Ex:

Arraylist<Integer> ls = new ArrayList<>();

Ex: import java.util.ArrayList;

```
class Eg1
{
    public static void main (String [] args)
    {
        ArrayList <Integer> ls = new ArrayList <>();
        ls.add(10);
        ls.add(20);
        ls.add(30);
        System.out.println(ls);
        // to access using get()
        Integer i = ls.get(1);
        System.out.println(i);
    }
}
```

o/p:

[10, 20, 30]

20

NOTE:

1) The elements are not converted to Object type, instead they will be converted for the given generic type.

2) The return type of the elements of generic collection will be same as the given generic type.

5) Whenever we go for non-generics it becomes mandatory to downcast the objects, since the return type of accessing methods will be of object type.

WAP to create a homogeneous ArrayList of Employees with 5 employee data in it. Remove an employee whose salary is equal to 10000Rs. [Use all the possible ways of accessing to do the task].

```
class Employee
{
Employee(){ }
int eid;
double sal;
String name;
Employee (int eid, String name, double sal)
{
this.eid = eid;
this.name = name;
this.sal = sal;
}
public String toString()
{
System.out.println("employee ID: " + eid);
System.out.println("Employee name: " + ename);
System.out.println("Salary: " + sal);
return "";
```

```
public boolean equals(Object o)
{
~~this.sal==~~
return (this.sal == 0);
}
}
import java.util.ArrayList;
class Driver
{
  public static void main(String[] args)
  {
    ArrayList<Employee> ls = new ArrayList<>();
    ls.add(new Employee(101,"sheela",500));
    ls.add(new Employee(102,"Leela",1000));
    ls.add(new Employee(103,"Mala",2000));
    ls.add(new Employee(104,"Laila",1500));
    ls.add(new Employee(105,"Mauasa",5000));
    //to access using get method.
    for(int i=0, i<ls.size(); i++)
    {
      Employee ~~Iterator~~ d = ls.get(i);
      if(d.sal.equals(1000))
      }
      ls.remove(d);
      }
    }
```

```
// to access using iterator method.

Iterator<Employee> s = ls.iterator();

while (s.hasNext())
{
    Employee e1 = s.next();
    if (e1.sal == 1000)
    {
        ls.remove(e1);
    }
}

// to access using listIterator method.

ListIterator<Employee> m = ls.listIterator();

while (m.hasNext())
{
    Employee e2 = m.next();
    if (e2.sal == 1000)
    {
        ls.remove(e2);
    }
}

// to access using advanced for loop // Cannot do using - for each loop

for (Employee e3 : ls)
{
    if (e3.sal == 100)
    {
        ls.remove(e3);
    }
}

}
```

NOTE:

In java, if we need to sort, the objects must be of type comparable, else we get class cast exception.

▶ TO SORT AN Arraylist:

To sort an Arraylist we can use sort() which present inside Collections class.

sort is a static method inside collections class. Therefore we can call it as below:

Collections.sort ()

sort is an overloaded method present inside collections class.

1. sort (Arraylist ls);
2. sort (Arraylist ls, Comparator ref)

sort (ArrayList ls):

* It sorts the Arraylist by default in ascending order
* It can sort the Arraylist only if the Arraylist is an homogeneous collection. else we get ClassCastException.

* The elements inside Arraylist must be of comparable type else we get ClassCastException.

Comparable interface:

Refer Arrays sorting for the notes.

```
import java.util.ArrayList;
import java.util.Collections;

class Marker Implements Comparable
{
 int id;
 String colour;
 double price;
 Marker(){}
 Marker(int id, String colour, double price)
  {
   this.id = id;
   this.colour = colour;
   this.price = price;
  }
 public String toString()
{
 System.out.println("ID : "+id + "; Colour: " + colour + "; Price."+
 return "";                                          price);
}
```

```
public int compareTo (Object o)
{
    Marker m = (Marker) o;
    if (this.id > m.id)
        return 1;
    else if (this.id < m.id)
        return -1;
    else
        return 0;
}

class Driver4
{
    public static void main (String[] args)
    {
        ArrayList ls = new ArrayList();
        ls.add (new Marker (101, "Black", 20));
        ls.add (new Marker (104, "Blue", 20));
        ls.add (new Marker (102, "Red", 20));
        ls.add (new Marker (103, "Green", 20));

        System.out.println("Before sorting : " + ls);
        //to sort
        Collection.sort(ls);
        System.out.println("After sorting : " + ls);
    }
}
```

➤ sort (Arraylist ls, ~~Comp~~ Comparator ref):

here, sort method accepts the two types of inputs

1) The list type

2) The comparator type as i/p.

➤ COMPARATOR

* Comparator is an interface present inside java.util package

* Inside Comparator interface we have one abstract~~or~~ method. Known as Compare method which can accept two arguments ie Compare (Object 01, Object 02)

NOTE:

for non-generic collections:

public int compare (Object 01, Object 02)

for generic collections:

public int compare (Type 1, Type 2)

To use the comparator ~~to~~ in order to sort the collect based on states:

STEP 1: Create a ~~cp~~ seperate class other than blue print & implement ~~it~~ Comparator interface.

Since Comparator interface has an abstract method, override the compare method present inside Comparato interface.

STEP 2 : Pass the comparator type reference to the sort method along with the list reference. ie for the sort method pass list type reference & create an object of class which implements -Comparator Interface.

Eg: import java.util.*;

```
Class Student
{
 int sid;
 String name;
 Student(){}
 Student (int sid, String name)
 {
 this.sid= sid;
 this.name = name;
 }
 public String toString()
 {
 S.oplu ("SID:" + sid + "; name:" + name );
 return "";
 }
```

```
class SortBySid implements Comparator
{
  public static void main (String[] args)
  {
    public int compare (Object o1, Object o2)
    {
      Student s1 = (Student) o1;
      Student s2 = (Student) o2;
      if (s1.sid > s2.sid)
          return 1;
      else if (s1.sid < s2.sid)
          return -1;
      else
          return 0;
    }
  }
class Driver
{
  public static void main (String[] args)
  {
    ArrayList ls = new ArrayList();
    ls.add (new Student (01, "Dinga"));
    ls.add (new Student (03, "Sheela"));
    ls.add (new Student (02, "Mala"));
    ls.add (new Student (04, "Laila"));
    S.opln ("Before sorting:" + ls);
```

```
Collection.sort(ls, new SortBysid());
So.plu("After sorting: " + ls);
}
}
```

SET :

Set is an interface, which is a sub-interface of collection interface. Therefore, all the methods of collection interface will be present inside Set interface.

Set is present inside java.util package.

characteristics :

1) It is an unordered Collection ie it doesn't maintain the order of insertion.

2) Set doesn't allow the duplicates.

3) It doesn't have indexing. Therefore we cannot access, insert or remove using index.

4) We can access the elements inside set only using two ways.

   i) using iterator()

   ii) for each loop

NOTE : The list of methods inside Set is same as that of list of methods present inside collection interface

▶ CONCRETE IMPLEMENTING CLASS OF Set :

We are going to discuss two important concrete Implementing classes of Set.

1) HashSet

2) TreeSet.

▶ HashSet :

* It is a concrete implementing class of Set.
* It has all the methods of collection interface

▶ Characteristics :

1) Unordered collection.

2) No duplicates

3) No indexing.

▶ CONSTRUCTORS :

1. HashSet() :- Creates an Empty Hashset Object

2) HashSet(Collection c) : creates an Hashset & copies all the object of Collection into the Hashset

Eg:

```
import java.util.HashSet;
class H1
{
  public static void main(String[] args)
  {
    HashSet hs = new HashSet();
    hs.add(10);
    hs.add(20);
    hs.add(30);
    hs.add(40);
    hs.add(20);
    System.out.println(hs);
  }
}
```

Op:

[20, 40, 30, 10]

Eg: To access the elements:

```
import java.util.HashSet;
class H2 {
public static void main(String[] args)
 {
  HashSet hs = new HashSet();
  hs.add("sheela");
```

```
hs.add("Leela");
hs.add("Laila");
hs.add("Urmila");
System.out.println(hs);
//to access using iterator.
Iterator i = hs.iterator();
while(i.hasNext())
{
System.out.println(i.next());
}
//to access using for each.
for(Object o : hs)
{
String s = (String) o;
System.out.println(s + " Length : " + s.length());
}
}
}
```

*I/P : [Leela, Laila, Urmila, Sheela]

```
Leela
Laila
Urmila
Sheela
Leela length : 5
Laila length : 5
Sheela length : 6
```

TreeSet:

It is also a concrete implementing class of Set. Therefore Treeset will have all the methods of Collection interface.

CHARACTERISTICS:

1) The elements will be sorted by default.

2) The elements to be added in Treeset must be of compa-le type else we get ClassCastException.

3) All the elements in Treeset should be of same ~~set~~ type else we get ClassCastException.

4) Duplicates are not allowed.

5) No indexing, therefore we cannot access the elements using indexing

CONSTRUCTORS:

1. Treeset(): It creates an empty Treeset object.

2. Treeset(Collection c): It creates a Treeset Object & copies all the elements of Collection into Treeset.

NOTE: Inside TreeSet we shouldn't store null as the data. If we try to do it then we get NullPointerExcepti

To access the elements of TreeSet in descending order

descendingIterator(): We can access the elements of TreeSet in descending order by creating an iterator with the help of descendingIterator().

Eg:-

```
impon java.util.Treeset;
class TS2{
public static void main (String[] args)
{
 Treeset ts= new Treeset();
 ts.add("sheela");
 ts.add("Leela");
 tc.add("Laila");
 ts.add("Urmila");
 System.out.println(ts);
 //to access using iterator.
 Iterator i= ts.iterator();
 while(i.hasNext())
  {
    S.opln (i.next());
  }
```

// to access the elements in descending order/reverse order

```
Iterator d = ts.descendingIterator();
while (d.hasNext())
 {
  S.o.plu (d.next());
 }
}
}
```

o/p :

[Laila, Leela, Sheela, Urmila]

Laila
Leela
Sheela
Urmila
Urmila
Shella
Leela
Laila

Can we convert one type of collection into another type of collection?

Yes, we can with the help of parameterised constructors of respective collection.

Eg: import java.util.*;

```
class TS3 {
public static void main(String[] args)
{
ArrayList ls = new ArrayList();
ls.add(10);
ls.add(30);
ls.add(20);
ls.add(50);
ls.add(40);
System.out.println("List : " + ls);
// to convert list to hash set
Hashset hs = new Hashset(ls);
System.out.println("Hashset : " + hs);
// to convert list to Tree Set
TreeSet ts = new TreeSet(ls);
System.out.println("Tree Set : " + ts);
}
}
```

O/p :

List : [10, 30, 20, 50, 40]

Hash Set : [50, 20, 40, 10, 30]

Tree Set : [10, 20, 30, 40, 50]

MAP :

· Map is a data structure or collection which helps us store the data in the form of key-value pairs, where every value is associated with a unique key.

NOTE :

1. Key cannot be a duplicate value
2. One key can be associated with only one value
3. Maps helps us to access the values with the help of its associated keys.

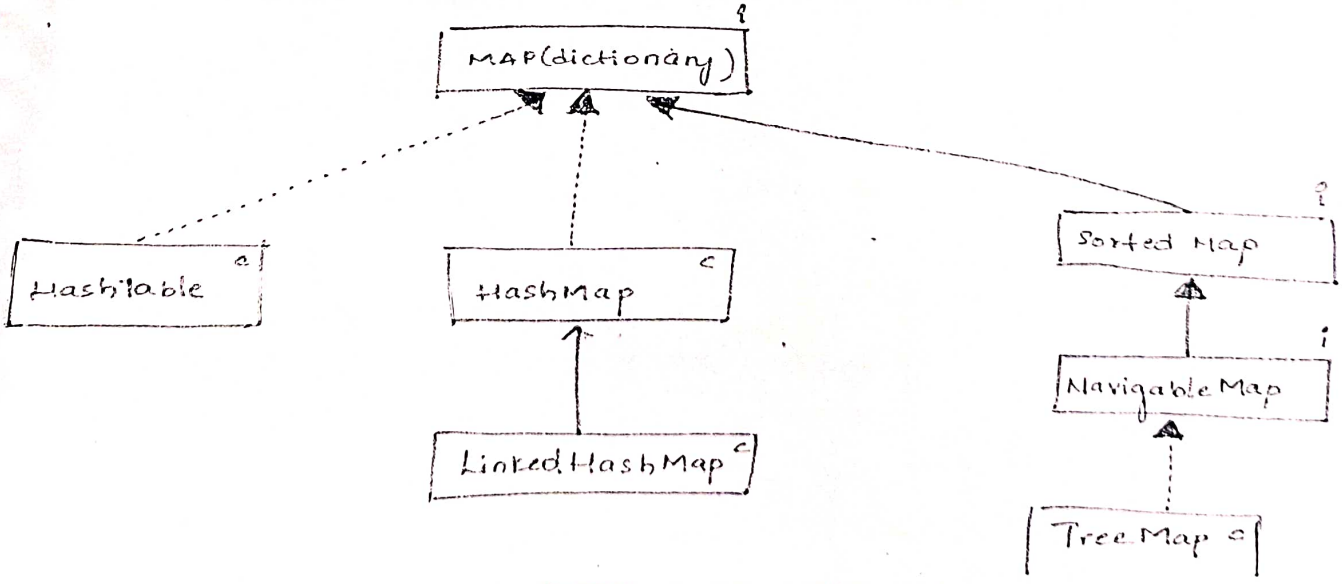
Ex 1 : 1) eid (key) associated with employee (value).

2) Reg number (key) associated with student (value).

Maps were called as dictionary.

~~HIERA~~

▶ HIERARCHY OF MAP



NOTE:

1. Map is an interface which is defined in java.util package.

2. We can create generic map by providing the type both key as well as value.

ie <key-type, value-type>, <K, V>.

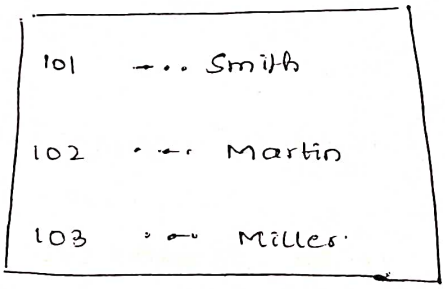
3) We can obtain 3 different views of a map.

a) We can obtain list of values from a map.

b) We can also obtain set of keys from a map.

c) We can obtain a set of key-value pairs.

Ex:



List of values : [Smith, Martin, Miller]

Set of keys : [101, 102, 103]

Set of key-value pairs: [101=Smith, 102=Martin, 103=Miller]

METHODS OF MAP INTERFACE:

SL NO	METHOD SIGNATURE	PURPOSE
1.	put(key, value)	1. Add an entry to the map 2. Replace the old value with a new value of an existing entry in the map.
2.	putAll(Map)	copies all the entries from the given ma into current map.
3.	containskey(key)	checks whether the key is present or not & returns a boolean value.
4.	containsValue(value)	checks whether the value is present or not & returns a boolean value.
5.	remove(key)	If the key is present, entry is removed from the map & the value is returned. If the key is not present, nothing is removed & null is returned.
6.	~~char~~ clear()	It removes all the entries in the map.
7.	get(key)	It returns the value for the specified key from the map, if the key is not present it will return null.
8.	isEmpty()	To check whether the map has mappings or not. The return type of this method is boolean.

	SIGNATURE	PURPOSE
9.	values( )	It returns a collection of values present in the map. Collection < V >.
10.	keySet( )	It returns a set of keys present in the map. Return type is Set < Key >
11.	entrySet( )	It returns a set of all the entries present in the map. Return type is Set < Key, value >
12.	size( )	It returns the size or number of entries inside the Map.

▶ **HashMap :**

It is the Concrete Implementing class of Map interface.

▶ **CHARACTERISTICS OF HashMap :**

1) The data is stored in the form of key-value pair.

2) The order of insertion is not maintained ie It is an unordered collection.

3) key & value can be null.

4) The key cannot be duplicate whereas values can be duplicate.

▶ Constructors :

1. HashMap() :

It creates an empty HashMap Object

2. HashMap(Map) :

It creates a HashMap Object & copies all the entry of the given map into the HashMap created.

▶ TreeMap :

It is a concrete implementing class of Map interface

▶ CHARACTERISTICS :

1. It also stores data in key-value pair.
2. It will sort the entries in the Map w.r.t key in ascending order.
3. The key in the TreeMap must be of type Comparable, if it is not Comparable type then we get ClassCastExcepti
4. In TreeMap key cannot be null, if it is null we get NullPointerException.
5. We can have value as null in TreeMap

▶ Constructors :

1) TreeMap() :

It creates an empty TreeMap Object

▶ TreeMap (Map) :

▷ It creates a TreeMap Object & copies all the entry of given Map into the TreeMap created.

▶ HashTable :

It is also a concrete Implementing class of Map interface. It can access the data as fast as possible in HashTable, there it uses hashing algorithm.

▶ CHARACTERISTICS :

1) It is used to store data in key-value format

2) In HashTable, the insertion order is not maintained.

3) In HashTable, both key & value cannot be null, if it is null we get NullPointerException.

▶ Constructors :

1). HashTable () :

It creates an empty HashTable Object.

2) HashTable (Map) :

It creates a HashTable Object & copies all the entry of the given Map into the HashTable created.

To convert an Array into Arraylist:

```
import java.util.Arraylist;

class s1
{
  public static Arraylist tolist (int[] a)
   {
    Arraylist ls= new Arraylist();
    for (int i=0; i<length; i++)
     {
        ls.add (a[i]);
     }
  }
  public static void main (String[] args)
   {
     int[]  a = {10,20,40}
     Array list l = toList (a);
     System.out.println (l);
   }
}.
```